

Java Programming

Mai Ngọc Khương
Hà nội, tháng 8 năm 2026

Câu hỏi 1

• Java là ngôn ngữ lập trình thuộc mô hình nào?

A. Procedural

B. Functional

C. Object-Oriented

D. Assembly

 Đáp án: C

Câu hỏi 2

Khẩu hiệu nổi tiếng của Java là gì?

- A. Code Once, Debug Forever
- B. Write Once, Run Anywhere
- C. Learn Once, Use Forever
- D. Run Anywhere, Compile Everywhere

 Đáp án: B

Câu hỏi 3

- Thành phần nào giúp Java chạy trên nhiều hệ điều hành khác nhau?

A. JDK

B. JRE

C. JVM

D. IDE



Đáp án: C

Câu hỏi 4

• Lệnh dùng để biên dịch file Java là gì?

A. java HelloWorld.java

B. javac HelloWorld.java

C. compile HelloWorld.java

D. run HelloWorld.java



Đáp án: B

Câu hỏi 5

• Đây là khai báo biến hợp lệ trong Java?

A. `int x;`

B. `integer x;`

C. `Int x;`

D. `number x;`

 Đáp án: A

Câu hỏi 6

• Phát biểu nào đúng về String trong Java?

A. Là kiểu dữ liệu nguyên thủy

B. Là kiểu tham chiếu

C. Là kiểu số thực

D. Là kiểu logic

 Đáp án: B

Câu hỏi 7

• Từ khóa dùng để khai báo hằng số là gì?

A. static

B. const

C. final

D. fixed

 Đáp án: C

Câu hỏi 8

- Giá trị của đoạn mã sau là gì?

```
int x = 5;
```

```
System.out.println(++x);
```

- A. 4
- B. 5
- C. 6
- D. Lỗi

☒ Đáp án: C (x tăng lên 6 trước khi in ra)

Câu hỏi 9

- Giá trị của đoạn mã sau là gì

```
int x = 10;
```

```
int y = x++;
```

```
System.out.println(y);
```

- A. 9
- B. 10
- C. 11
- D. Lỗi



Đáp án: B

Giải thích: `x++` là tăng sau (post-increment), nên giá trị của `x` được gán cho `y` trước, sau đó `x` mới tăng lên 11. Vì vậy `y = 10`.

Câu hỏi 10

- Giá trị in ra của đoạn mã sau:

```
int[] arr = {10, 20, 30, 0};
```

```
System.out.println(arr.length);
```

A. 3

B. 4

C. 5

D. Lỗi



Đáp án: B

Giải thích: Thuộc tính `length` trả về số phần tử của mảng. Mảng trên có 4 phần tử: 10, 20, 30 và 40

Câu hỏi 11

- Cho $x = y--$;

Sau khi câu lệnh được chạy, đáp án nào đúng?

A. $x < y$

B. $x > y$

C. $x == y$

 Đáp án: B

Câu hỏi 12

- Chương trình có biên dịch được không?

```
int iArray[] = new int[] {1,2,3,4,5};
```

A.Có

B.Không

 Đáp án: A

Convert kiểu data

Convert kiểu là gì

■ Convert kiểu là

Convert một kiểu data thành một kiểu data khác.

● Ví dụ cần thiết phải convert kiểu

● Convert kiểu chuỗi ký tự (String)

strUserEntry xxxx → xxxx

Gán

● Convert kiểu số nguyên (int)

intAmount

Error

"5250"

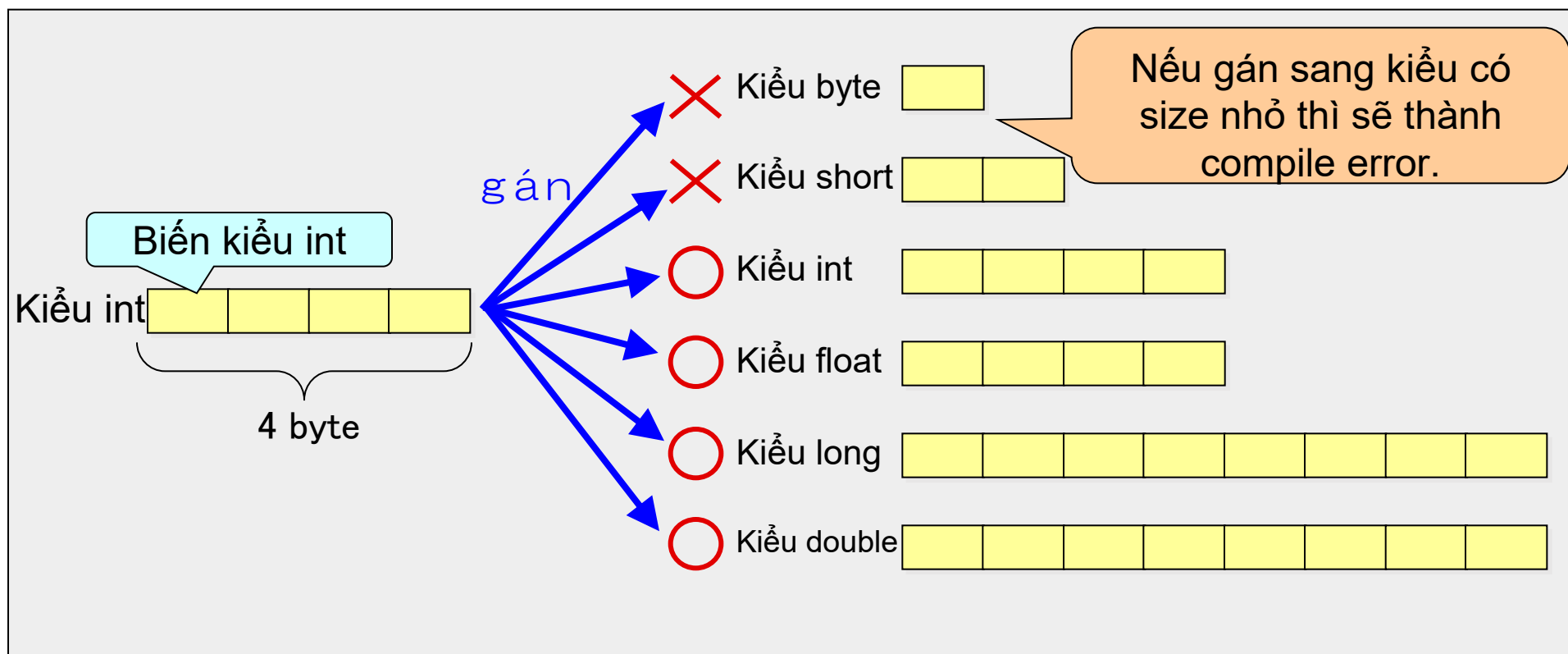
Compile error do 「kiểu không tương thích」

Convert kiểu không tường minh (1)

■ Convert kiểu không tường minh là gì

Khi tính toán và gán giá trị thì việc convert sẽ được tiến hành tự động

● Ví dụ có thể và không thể convert ngầm (trường hợp kiểu int)



Convert kiểu không tường minh (2)

● Ví dụ về convert kiểu không tường minh

```
int valInt = 4;
```

```
byte valByte = valInt;
```

```
long valLong = valInt;
```

```
double valDouble = valInt;
```

Ví dụ bị compile error

Ví dụ có thể convert kiểu không tường minh

● Ví dụ convert khi tính toán

```
double valDouble2 = 10 / 4;
```

```
double valDouble3 = 10 / 4d;
```

Sau khi tính toán bằng kiểu int thì gán sang kiểu double

Tính toán sau khi convert thành kiểu double rồi gán sang kiểu double

Convert kiểu tường minh (Cast) (1)

■ Convert kiểu tường minh (cast) là gì

Sử dụng trong trường hợp không thể convert không tường minh.v.v...

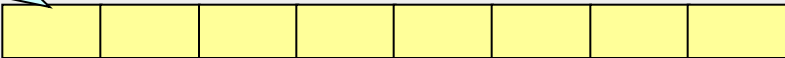
Convert cưỡng chế một kiểu data thành một kiểu data khác.

● Ví dụ convert kiểu tường minh

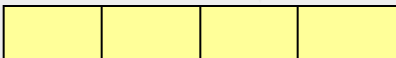
Tên biến kiểu int = Tên biến kiểu (int) double;

Toán tử cast

Biến kiểu double

Kiểu double  (8 byte)

Gán

Kiểu int  (4 byte)

Cần phải chú ý vì giá trị có thể bị mất

Convert kiểu tương minh (Cast) (2)

■ Convert kiểu tương minh (cast)

● Cú pháp cơ bản

Toán tử cast

(Kiểu data mới) giá trị hoặc biểu thức

● Ví dụ

```
double valDouble = 1.6;
```

```
int valInt1 = (int) valDouble ;
```

Cast 1.6 thành kiểu int → Phần sau dấu phẩy bị cắt

```
int valInt2 = (int) (2 * 1.6) ;
```

Sau khi thực hiện phép nhân thì cast thành kiểu int → Phần sau dấu phẩy bị cắt

※ Giá trị của valInt1 thành 「1」 và giá trị của valInt2 thành 「3」.

Convert từ chuỗi ký tự thành giá trị số (1)

■ Convert từ chuỗi ký tự thành giá trị số

Sử dụng **parseXXX** của **wrapper class** để sử dụng kiểu data cơ bản như một object.

(**XXX** là chuỗi ký tự thể hiện kiểu data)

Integer class của kiểu
int.v.v...

● Ví dụ convert từ chuỗi ký tự thành giá trị số

● Convert kiểu chuỗi ký tự (String)

strUserEntry

xxxx

xxxx

"5250"

● Convert kiểu số nguyên (int)

intAmount

5250

convert

Sử dụng method **parseInt** của
Integer class để convert
Integer.parseInt(strUserEntry)

Convert từ chuỗi ký tự thành giá trị số (2)

■ Convert từ chuỗi ký tự thành giá trị số

● Cú pháp cơ bản

`parseInt, parseDouble v.v...`

Wrapperclassname.parseXXX (chuỗi ký tự)

● Ví dụ

```
String str1 = "17";
```

```
String str2 = "2.6";
```

```
int valInt = Integer.parseInt(str1);
```

```
long valLong = Long.parseLong("1200");
```

```
double valDouble = Double.parseDouble(str2);
```

Chuỗi ký tự sẽ truyền cho
parseXXX là biến số hay literal đều
được.

2.6

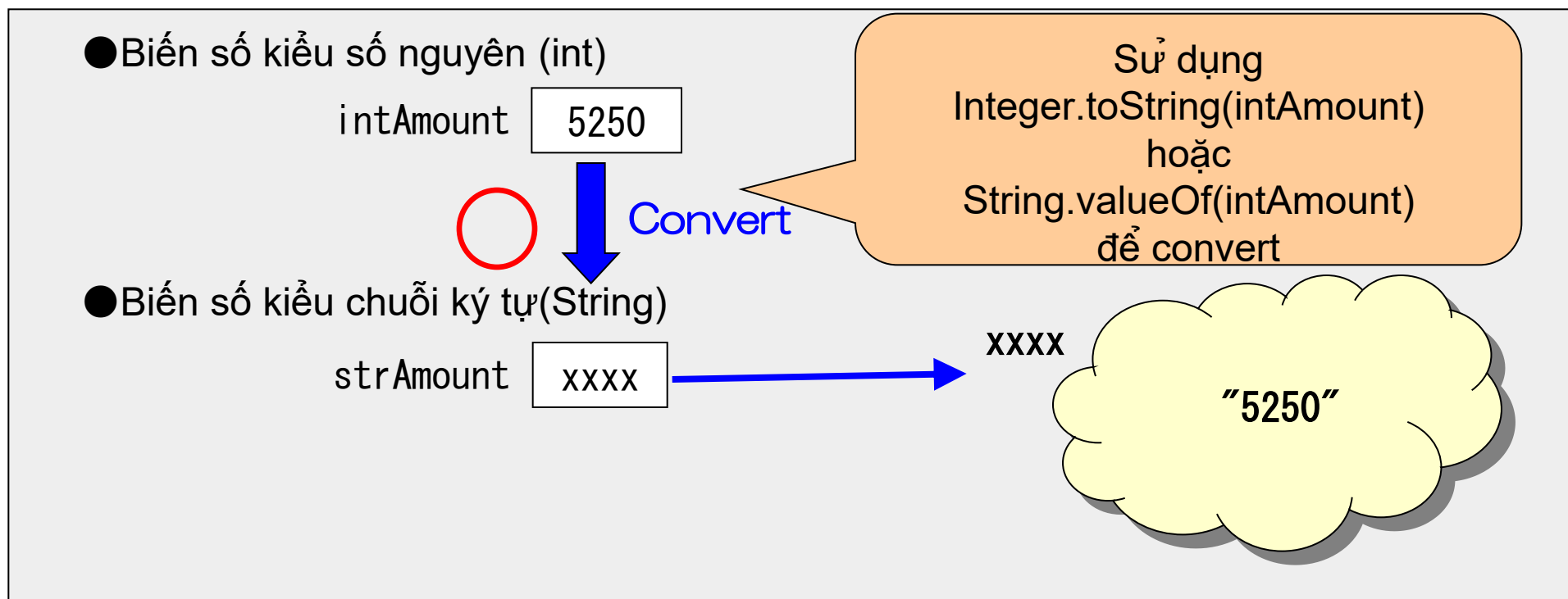
"2.6"

Convert từ giá trị số thành chuỗi ký tự (1)

■ Convert từ giá trị số thành chuỗi ký tự

Wrapperclassname.toString method, hoặc, sử dụng String.valueOf method.

● Ví dụ



Convert từ giá trị số thành chuỗi ký tự (2)

■ Convert từ giá trị số thành chuỗi ký tự

● Cú pháp cơ bản

Integer, Double v.v...

Wrapperclassname.toString(Giá trị số của kiểu data cơ bản của wrapper class.)

String.valueOf(Giá trị số)

● Ví dụ

```
String str1 = Integer.toString(12);  
int valInt = 123;  
String str3 = Integer.toString(valInt);
```

Giá trị số sẽ truyền vào
toString là biến số, literal đều
được.

```
String str2 = String.valueOf(12);  
long valLong = 570000;  
String str4 = String.valueOf(valLong);
```

Giá trị số sẽ truyền vào
valueOf là biến số, literal
đều được.

"570000"

570000

Câu lệnh control

Câu lệnh control là gì

■ Câu lệnh control là

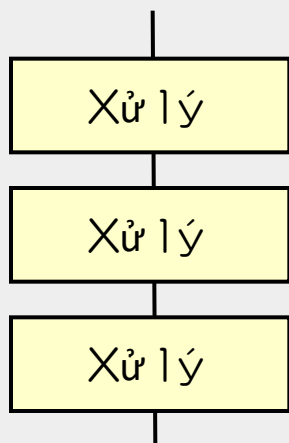
Cú pháp để control flow xử lý của program.

Trường hợp phân nhánh xử lý theo điều kiện thì sẽ sử dụng ví dụ như câu lệnh if.

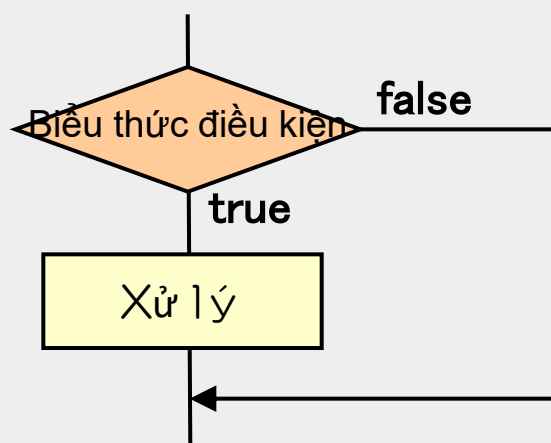
Trường hợp thực hiện xử lý lặp lại thì sẽ sử dụng ví dụ như câu lệnh while và câu lệnh for.

■ 3 cấu trúc cơ bản

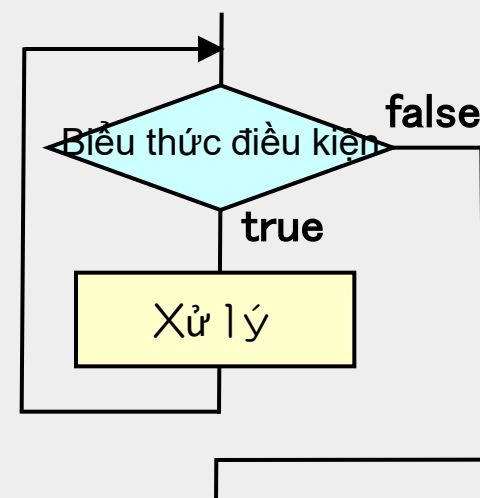
Cấu trúc tuần tự



Cấu trúc phân nhánh



Cấu trúc lặp

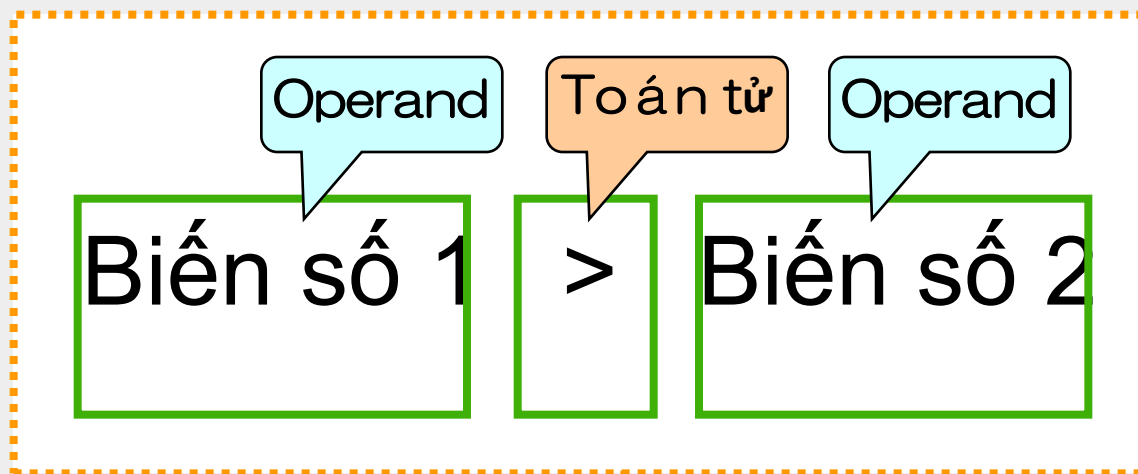


Biểu thức điều kiện

■ Biểu thức điều kiện là gì

Là biểu thức mà kết quả đánh giá là true hoặc false.

● Ví dụ của biểu thức điều kiện



↓

Biểu thức điều kiện.

① Biểu thức điều kiện được thỏa mãn. **true (đúng)**

② Biểu thức điều kiện không được thỏa mãn. **false (sai)**

Toán tử quan hệ và biểu thức điều kiện.

■ Toán tử quan hệ.

> >= < <= == !=

● Ví dụ về biểu thức điều kiện đã sử dụng toán tử quan hệ.

Toán tử quan hệ	Ví dụ về biểu thức điều kiện	Ý nghĩa của biểu thức điều kiện.
>	$A > B$	A lớn hơn B
>=	$A \geq B$	A lớn hơn hoặc bằng B
<	$A < B$	A nhỏ hơn B
<=	$A \leq B$	A nhỏ hơn hoặc bằng B
==	$A == B$	A bằng B
!=	$A != B$	A không bằng B

Toán tử logic và biểu thức điều kiện

■ Toán tử logic

&& || !

● Toán tử logic và biểu thức điều kiện.

Toán tử logics	Cách sử dụng	Ý nghĩa của biểu thức điều kiện	Điều kiện
&&	Biểu thức điều kiện A && biểu thức điều kiện B	Thỏa mãn biểu thức điều kiện A và biểu thức điều kiện B.	AND
	Biểu thức điều kiện A biểu thức điều kiện B	Thỏa mãn biểu thức điều kiện A hoặc biểu thức điều kiện B	OR
!	! Biểu thức điều kiện A	Không thỏa mãn biểu thức điều kiện A.	NOT

【Kết quả đánh giá đã sử dụng toán tử logic đối với biểu thức điều kiện A, B】

Điều kiện AND		Biểu thức điều kiện B	
		true	false
Biểu thức điều kiện A	true	true	false
	false	false	false

Điều kiện OR		Biểu thức điều kiện B	
		true	false
Biểu thức điều kiện A	true	true	true
	false	true	false

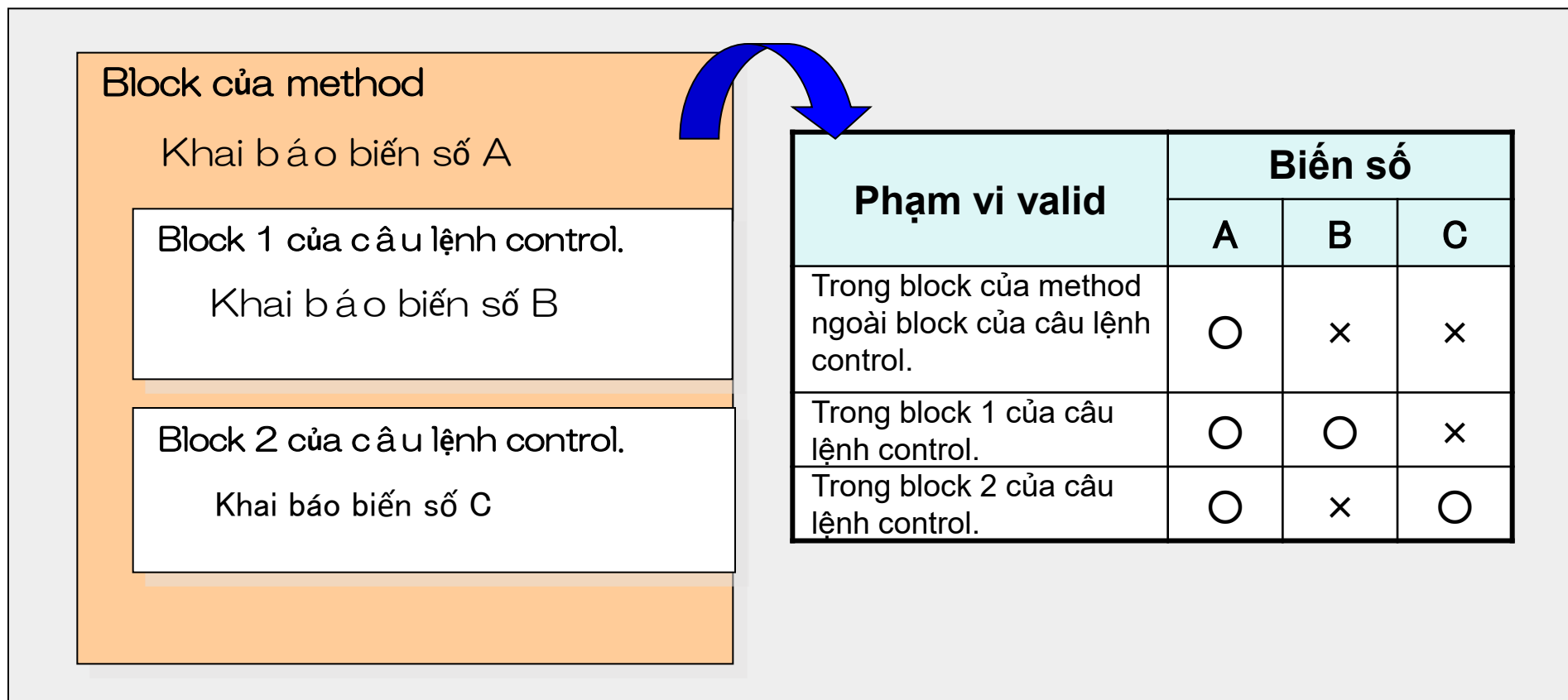
Điều kiện NOT		Đánh giá
Biểu thức điều kiện A	true	false
	false	true

Câu lệnh control và phạm vi valid của biến số

■ Phạm vi valid của biến local

Trong block đã được khai báo.

● Nơi khai báo biến số.



Kiểm soát phân nhánh điều kiện

Kiểm soát phân nhánh điều kiện là gì

■ Kiểm soát phân nhánh điều kiện là

Câu lệnh kiểm soát sẽ mô tả trong method

Suy nghĩ chu trình xử lý theo điều kiện

■ Câu lệnh kiểm soát của phân nhánh điều kiện

● Câu lệnh if

Có thể chỉ định biểu thức điều kiện của số tùy ý, và phân nhánh xử lý theo biểu thức điều kiện

- Câu lệnh if
- Câu lệnh if-else
- Câu lệnh if-else if-else

● Câu lệnh switch

Phân nhánh xử lý theo giá trị của kết quả đánh giá của một biểu thức điều kiện

Câu lệnh if

■ Câu lệnh if

● Cú pháp cơ bản

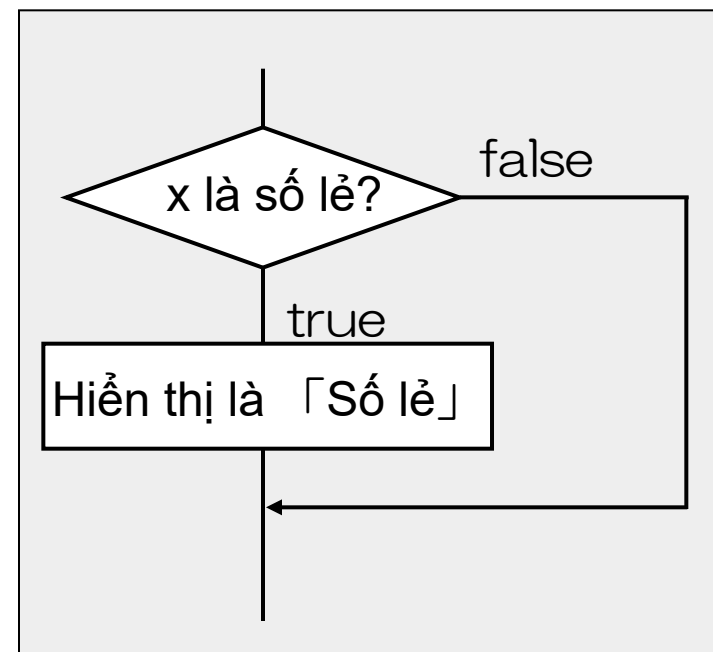
if (Biểu thức điều kiện) {

Xử lý trong trường hợp thỏa mãn biểu thức điều kiện}

● Ví dụ

```
int x = 5;
```

```
if (x % 2 == 1) {  
    System.out.println("Số lẻ");  
}
```



Câu lệnh if-else

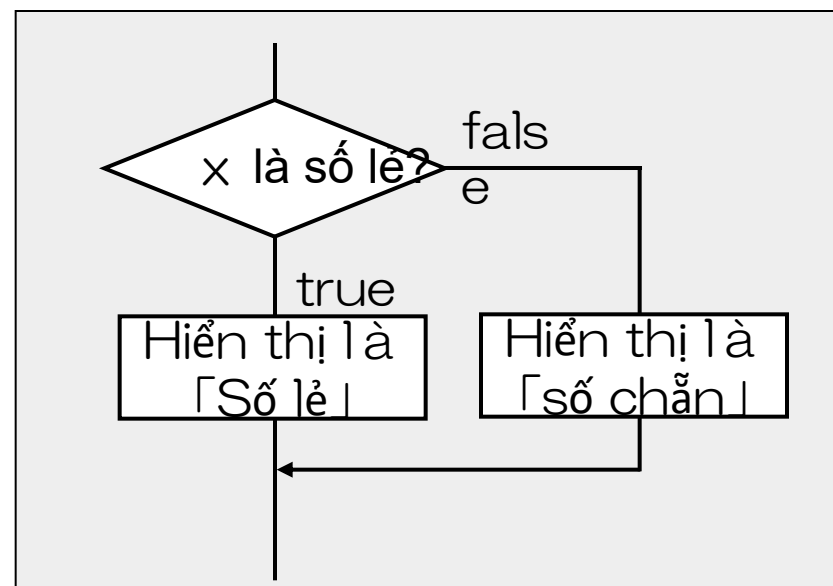
■ Câu lệnh if-else

● Cú pháp cơ bản

```
if (Biểu thức điều kiện) {  
    Xử lý trong trường hợp đã thỏa mãn biểu thức điều kiện  
} else {  
    Xử lý trong trường hợp không thỏa mãn biểu thức điều kiện  
}
```

● Ví dụ

```
int x = 4;  
  
if (x % 2 == 1) {  
    System.out.println("Số lẻ");  
} else {  
    System.out.println("Số chẵn");  
}
```



Câu lệnh if-else if-else(1)

■ Câu lệnh if-else if-else

● Cú pháp đơn giản

```
if (Biểu thức điều kiện 1) {
```

Xử lý trong trường hợp đã thỏa mãn biểu thức điều kiện 1

```
}
```

```
else if (Biểu thức điều kiện 2) {
```

Xử lý trong trường hợp thỏa mãn biểu thức điều kiện 2

```
}
```

```
else {
```

Xử lý trường hợp không thỏa mãn cả hai biểu thức điều kiện 1, 2

```
}
```

Block else if sử dụng trong trường hợp muốn chỉ định ≥ 3 vị trí phân nhánh (Có thể mô tả nhiều)

Nếu không cần xử lý trường hợp không thỏa mãn tất cả biểu thức điều kiện thì có thể lược bỏ block else.

Câu lệnh if-else if-else (2)

■ Câu lệnh if-else if-else

- Ví dụ: Nếu $x < 0$ thì sẽ hiển thị 「số âm」, nếu là 0 thì sẽ hiển thị là 「0」, ngoài ra sẽ hiển thị là 「số dương」

```
int x = 5;
```

```
if ( x < 0 ) {
```

```
    System.out.println("Số âm");
```

```
} else if ( x == 0 ) {
```

```
    System.out.println("0");
```

```
} else {
```

```
    System.out.println("Số dương"); // Hiển thị số này
```

```
}
```

Trong ví dụ này là false

Trong ví dụ này
là false

Nesting của câu lệnh if (Lồng) (1)

■ Nest của câu lệnh if (lồng)

● Ví dụ

```
if (Biểu thức điều kiện 1) {
```

```
    if (Biểu thức điều kiện 2) {
```

```
        Xử lý khi thỏa mãn biểu thức điều kiện 2
```

```
    }
```

```
} else {
```

```
    if (Biểu thức điều kiện 3) {
```

```
        Xử lý khi thỏa mãn biểu thức điều kiện 3
```

```
    } else {
```

```
        Xử lý trong trường hợp không thỏa mãn  
        biểu thức điều kiện 3
```

```
    }
```

Có thể nesting câu lệnh if trong câu lệnh if đã nesting

Nest của câu lệnh if

Trường hợp block bị lồng, nếu xuống dòng (indent) thì code sẽ dễ nhìn.

Nest của câu lệnh if (lồng) (2)

■ Nest của câu lệnh if (lồng)

- Ví dụ: Biểu thị việc x và y có lớn hơn 0 không?

```
int x = 5;
int y = -2;
if (x > 0) {
    if (y > 0) {
        System.out.println("Cả x và y đều > 0");
    } else {
        System.out.println("x > 0, và y <= 0"); //Hiển thị câu này
    }
} else {
    if (y > 0) {
        System.out.println("x <= 0, và y > 0");
    } else {
        System.out.println("Cả x và y đều <= 0");
    }
}
```

Trong VD này là true

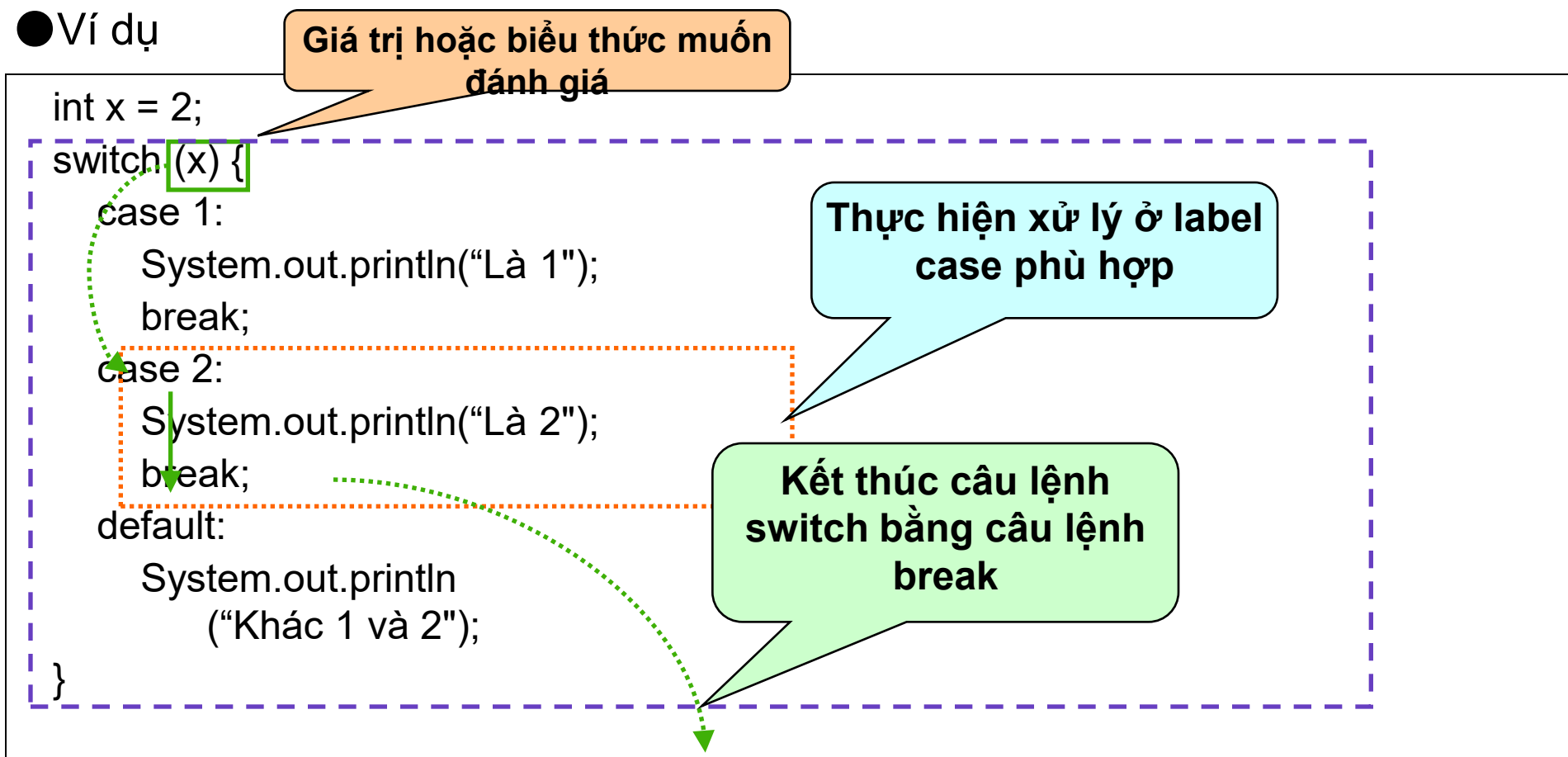
Trong VD này là false

Câu lệnh switch

■ Câu lệnh switch là

Là câu lệnh kiểm soát phân nhánh chu trình xử lý thành nhiều nhánh tùy theo kết quả đánh giá của 1 biểu thức điều kiện

● Ví dụ



Câu lệnh switch có chứa câu lệnh break

■ Câu lệnh switch chứa câu lệnh break

● Cú pháp cơ bản

Giá trị hoặc biểu thức có thể convert ngầm thành kiểu int

switch (Giá trị hoặc biểu thức muốn đánh giá) {

case Biểu thức hằng số 1:

Literal hoặc biểu thức mà kết quả đánh giá là literal

Xử lý của trường hợp giá trị hoặc biểu thức muốn đánh giá bằng với biểu thức hằng số 1

break;

Thoát khỏi switch block

:

case Biểu thức hằng số n:

Xử lý của trường hợp giá trị hoặc biểu thức muốn đánh giá bằng biểu thức hằng số n

break;

default:

Xử lý của trường hợp không phù hợp với toàn bộ case

Tương ứng với block else của câu lệnh if

}

Câu lệnh switch không chứa câu lệnh break (1)

■ Câu lệnh switch không chứa câu lệnh break

● Cú pháp cơ bản

Trường hợp thỏa mã điều kiện nào đó thì sẽ thực hiện cả xử lý của case phía sau đó vô điều kiện

switch (Giá trị hoặc biểu thức muốn đánh giá) {

case Biểu thức hằng số 1:

Literal hoặc biểu thức có kết quả đánh giá là literal

Xử lý của trường hợp giá trị hoặc biểu thức muốn đánh giá bằng biểu thức hằng số 1

:

case Biểu thức hằng số n:

Xử lý của trường hợp giá trị hoặc biểu thức muốn đánh giá bằng biểu thức hằng số n.

default:

Xử lý của trường hợp không khớp với toàn bộ case

Tương ứng với block else của câu lệnh if

}

Câu lệnh switch không chứa câu lệnh break (2)

■ Câu lệnh switch không chứa câu lệnh break

- Ví dụ : Dự định nếu x là 1 thì hiển thị là 「1」, nếu là 2 thì hiển thị là 「2」, ngoài ra thì hiển thị là 「khác 1, 2」, nhưng lại quên viết câu lệnh break .

```
int x = 2;
```

```
switch (x) {
```

```
case 1:
```

```
    System.out.println("1");
```

```
case 2:
```

```
    System.out.println("2");
```

```
default:
```

```
    System.out.println("Khác 1, 2");
```

```
}
```

Vì không có câu lệnh break nên sẽ tiếp tục thực hiện xử lý phía sau label case thỏa mãn.

● Ví dụ thực hiện

2

Khác 1, 2

Điều kiện OR và câu lệnh switch (1)

■ Điều kiện OR và câu lệnh switch

● Cú pháp cơ bản

switch (Giá trị hoặc biểu thức muốn đánh giá) {

case Biểu thức hằng số 1:

case Biểu thức hằng số 2:

:

case Biểu thức hằng số n:

Chỉ định liên tục case biểu thức hằng số :
➡ Điều kiện OR

Giá trị hoặc biểu thức muốn đánh giá bằng biểu thức hằng số

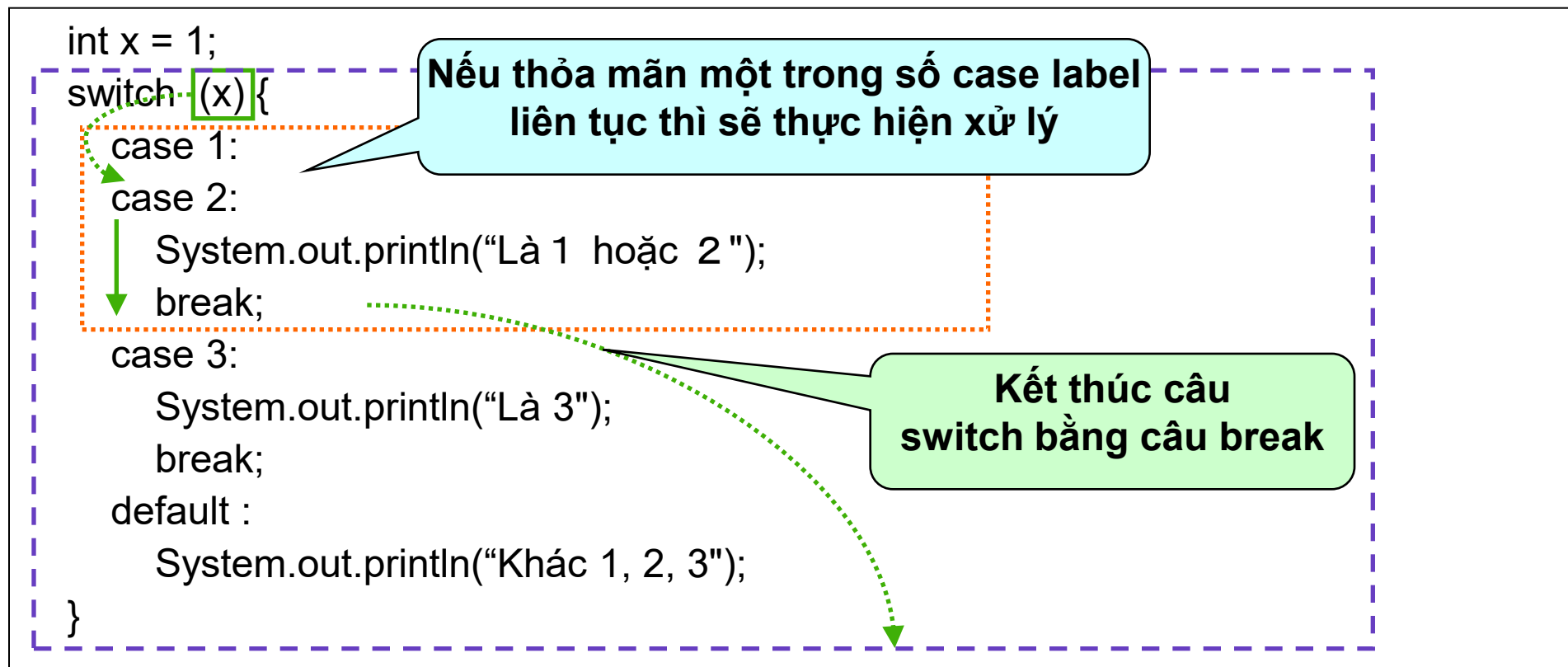
}

Tùy theo case, có thể tồn tại cả loại có chỉ định câu lệnh break và loại không chỉ định

Điều kiện OR và câu switch (2)

■ Điều kiện OR và câu lệnh switch

- Ví dụ: Nếu x là 1 hoặc 2 thì sẽ hiển thị là 「Là 1 hoặc 2」



- Ví dụ thực hiện

Là 1 hoặc 2

Control lắp

Outline

- Câu lệnh while
- Câu lệnh do-while
- Câu lệnh for

Control lặp là gì

■ Control lặp là

Câu lệnh control, sẽ mô tả trong method

Control để thực hiện xử lý lặp giống nhau

■ Các loại của câu lệnh control lặp

● Câu lệnh while

Thực hiện xử lý lặp trong khi biểu thức điều kiện là true (phán đoán trước điều kiện)

● Câu lệnh do-while

Thực hiện xử lý lặp trong khi biểu thức điều kiện là true (phán đoán sau điều kiện)

● Câu lệnh for(1) : Câu lệnh for

Chỉ định bắt đầu và kết thúc của xử lý lặp, thực hiện xử lý lặp

● Câu lệnh for(2) : Câu lệnh foreach

Thực hiện xử lý lặp đối với element của mảng hoặc collection

Câu lệnh while

■ Câu lệnh while

● Cú pháp cơ bản

Phán đoán trước điều kiện

```
while (Điều kiện lặp) {  
    Xử lý muốn lặp khi điều kiện là true  
}
```

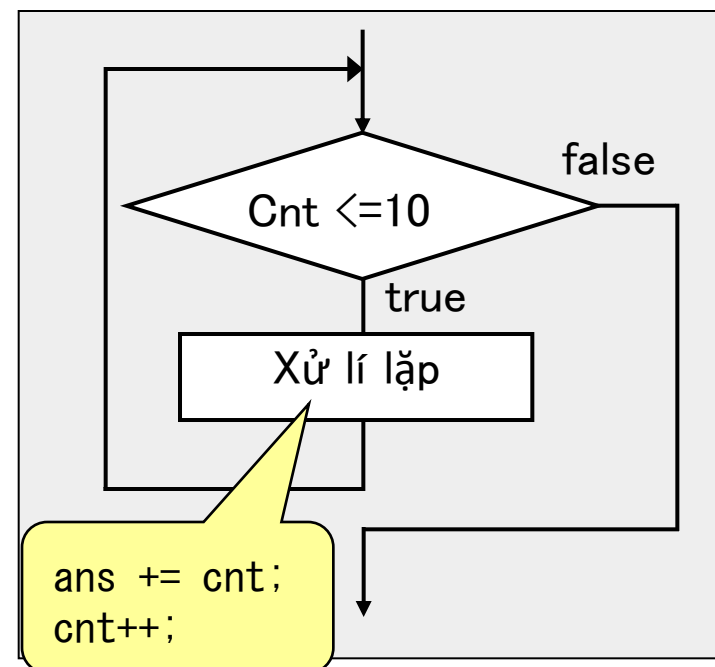
● Ví dụ

```
int cnt = 1;  
int ans = 0;
```

```
while (cnt <= 10) {  
    ans += cnt;  
    cnt++;  
}
```

```
System.out.println(cnt + ", " + ans);
```

Xử lý để thay đổi điều kiện lặp từ true sang false



Câu lệnh while



Chú ý

Gọi xử lý lặp không bao giờ kết thúc là lặp vô hạn
Để không tạo ra lặp vô hạn thì trong xử lý lặp của câu lệnh while phải có xử lý thay đổi điều kiện lặp chuyển từ true sang false .

(Ở ví dụ trên là `cnt++;`)

Hãy mô tả cẩn thận sao cho không thành loop vô hạn

Câu lệnh do-while

■ Câu lệnh do-while

● Cú pháp cơ bản

```
do {
    Xử lý muốn lặp khi điều kiện là true
} while (Điều kiện lặp);
```

Cần có ; ở cuối cùng của câu lệnh do-while

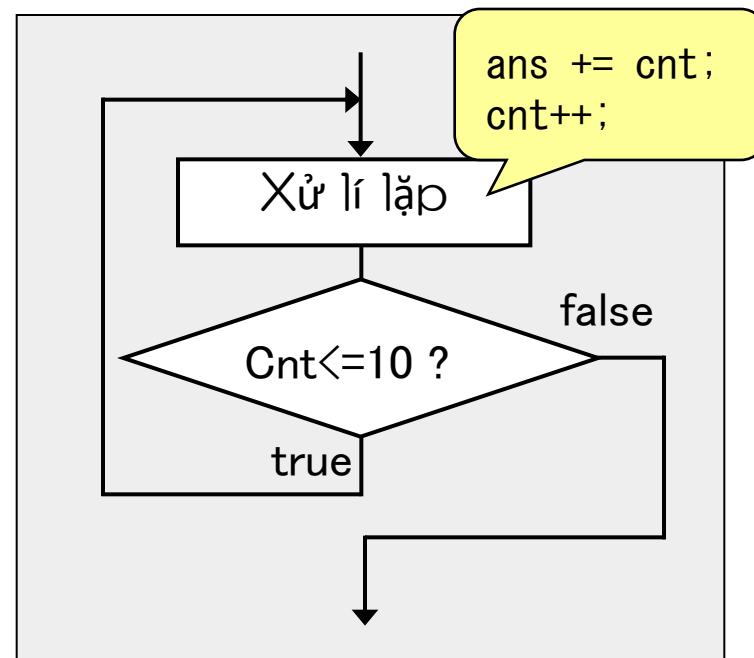
● Ví dụ

Phán đoán sau điều kiện

```
int cnt = 1;
int ans = 0;
do {
    ans += cnt;
    cnt++;
} while (cnt <= 10);
```

Xử lý để thay đổi điều kiện lặp từ true sang false

```
System.out.println(cnt + ", " + ans);
```



Bảng kết quả

- Tóm tắt quan hệ của điều kiện lặp và giá trị các biến số tại ví dụ trước thành bảng thì ta có bảng như sau
- Các giá trị là giá trị khi phán đoán điều kiện lặp

Phán đoán	Biến số cnt	Điều kiện lặp (cnt <= 10)	Biến số ans
Lần thứ 1	1	true	0
Lần thứ 2	2	true	1
Lần thứ 3	3	true	3
Lần thứ 4	4	true	6
Lần thứ 5	5	true	10
Lần thứ 6	6	true	15
Lần thứ 7	7	true	21
Lần thứ 8	8	true	28
Lần thứ 9	9	true	36
Lần thứ 10	10	true	45
Lần thứ 11	11	false	55

Câu lệnh for (1)

■ Câu lệnh for

● Cú pháp cơ bản

Nếu xử lí lặp được thực hiện 1 lần thì sẽ tự động tăng giảm biến số counter với số đã được chỉ định

for (setting giá trị khởi tạo của biến số counter; Biểu thức điều kiện; tăng giảm biến số counter) {

Xử lí muốn lặp khi điều kiện là true

}

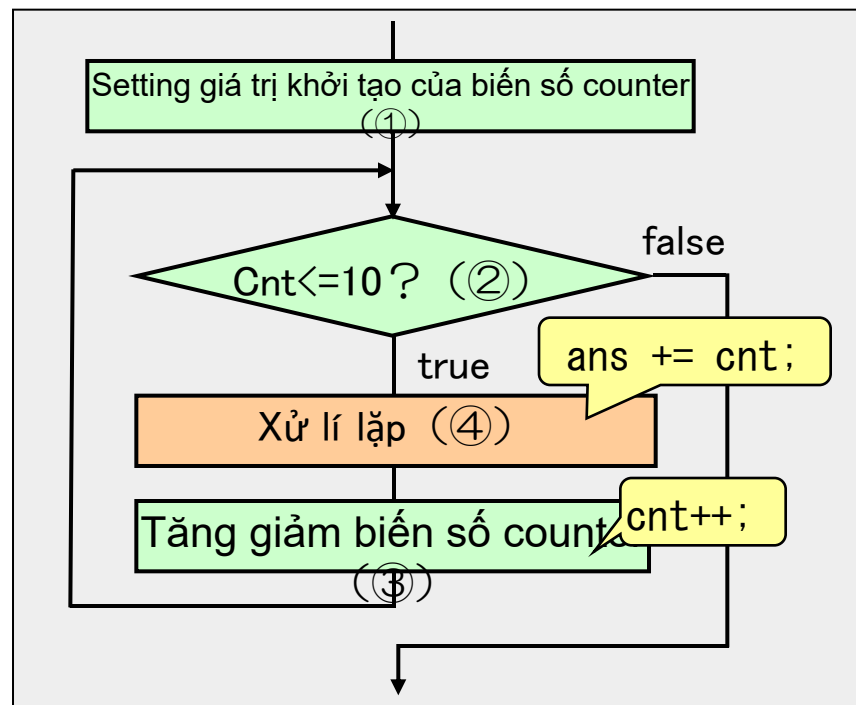
● Ví dụ

Xử lí để thay đổi điều kiện lặp từ true sang false

```
int ans = 0;
```

```

    ①
    ↓
for ( int cnt=1;    ②    cnt<=10;    ③    cnt++ ) {
    ans += cnt;    ④
}
System.out.println(ans);
  
```



Bảng kết quả

- Tóm tắt quan hệ của điều kiện lặp và giá trị các biến số tại ví dụ trước thành bảng thì ta có bảng như sau
- Các giá trị là giá trị khi phán đoán điều kiện lặp

Phán đoán	Biến số cnt	Điều kiện lặp (cnt <= 10)	Biến số ans
Lần 1	2	true	1
Lần 2	3	true	3
Lần 3	4	true	6
Lần 4	5	true	10
Lần 5	6	true	15
Lần 6	7	true	21
Lần 7	8	true	28
Lần 8	9	true	36
Lần 9	10	true	45
Lần 10	11	false	55

Câu lệnh for (2)

■ Câu lệnh foreach

● Cú pháp cơ bản

Đưa ra element của mảng or collection vào từng biến số một trong mỗi lần loop và thực hiện xử lý

```
for (Kiểu của element mảng tên biến số : mảng hoặc collection) {  
    xử lý muốn lặp đối với các element  
}
```

● Ví dụ

```
String[] strArray =  
    {"Smith", "Brown", "Allen"};
```

Đưa ra element của mảng vào từng str một

```
for (String str : strArray) {  
    System.out.println(str);  
}
```

Khai báo mảng strArray và khởi tạo

Loop hiển thị
str = "Smith", "Brown", "Allen"

Hiển thị str

Loop hiển thị

Control lặp và nest

■ Control lặp và nest

- Ví dụ : Hiển thị tổ hợp giá trị x và y đã sử dụng từ 1 đến 3

```
for (int x = 1; x <= 3; x++) {
```

```
    for (int y = 1; y <= 3; y++) {  
        System.out.println("(x,y):" + x + ", " + y);  
    }
```

```
    System.out.println("-----");
```

```
}
```

Loop phía ngoài

Loop bên trong

Trường hợp block là nest, nếu thực hiện indent thì code dễ đọc hơn

Control lặp và câu lệnh break

■ Control lặp và câu lệnh break

● Cú pháp cơ bản

break;

Câu lệnh break sẽ dùng trong trường hợp muốn thoát ra khỏi xử lý lặp

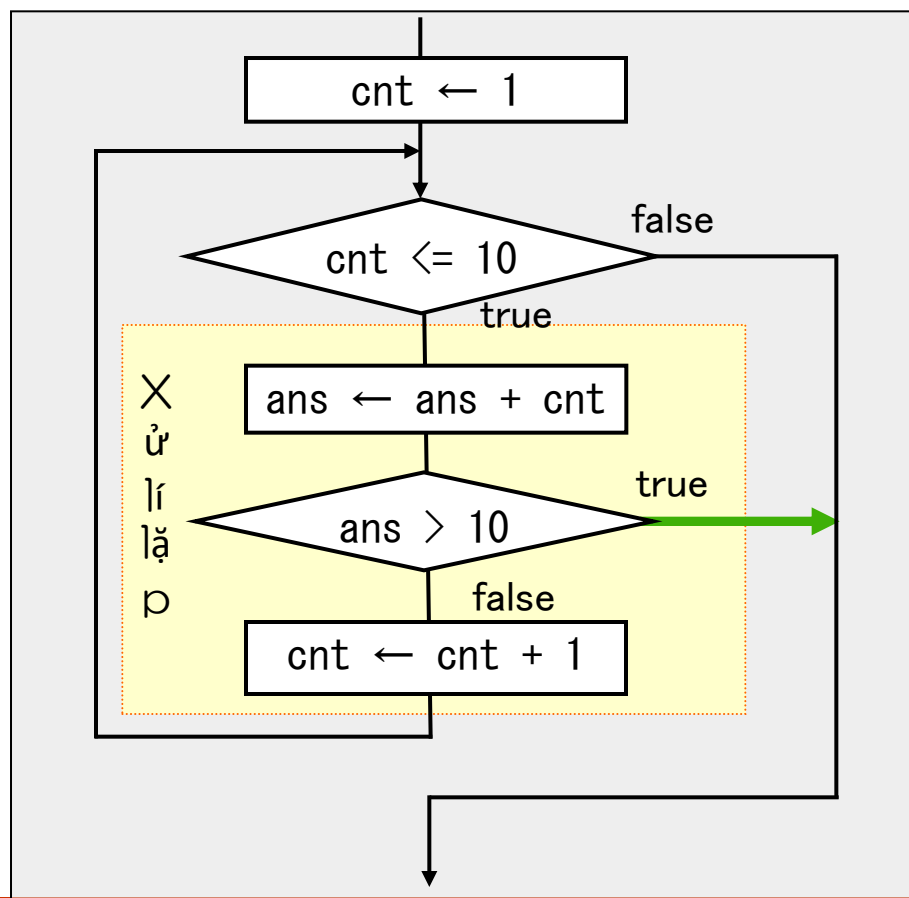
● Ví dụ

```
int ans = 0;
```

```
for (int cnt = 1; cnt <= 10; cnt++) {
    ans += cnt;
    if (ans > 10) {
        break;
    }
}
```

Thoát câu lệnh for

```
System.out.println(ans);
```



Control lặp và câu lệnh continue

■ Control lặp và câu lệnh continue

● Cú pháp cơ bản

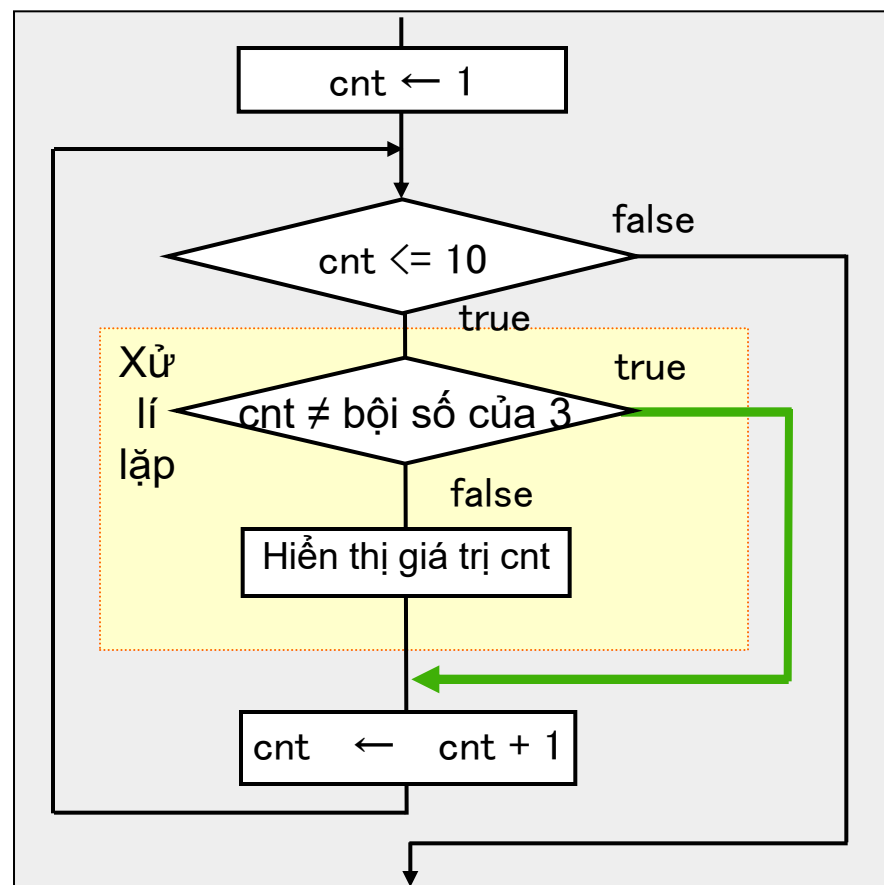
Câu lệnh continue, sẽ dùng khi skip xử lý còn lại của xử lý lặp

continue;

● Ví dụ

```
for (int cnt = 1; cnt <= 10; cnt++) {  
    if ((cnt % 3) != 0) {  
        continue;  
    }  
    System.out.println  
        ("Bội số của 3 : " + cnt);  
}
```

Skip xử lý
còn lại



Câu hỏi:

- Giả sử tất cả các biến đều được khai báo và khởi tạo đúng cách, chương trình sau có biên dịch được không?

```
for ( i = 1, j = 2; i < 3; i++, j++ ){  
};
```

A. Có

B. Không

☒ Đáp án: A

Câu hỏi:

Cast có phải là một từ khóa trong Java?

A. Có

B. Không

☒ Đáp án: B

Câu hỏi:

Trong vòng lặp for, câu lệnh continue sẽ thoát ra khỏi vòng lặp và tiếp tục tại câu lệnh đầu tiên dưới vòng lặp?

A. Đúng

B. Sai

☒ Đáp án: B